Sizes of Data Types and Code Suffix

C Decleration	Intel Data Type	Assembly Code Suffix	Size (Bytes)
char	Byte	b	1
short	Word	w	2
int	Double word	l	4
long	Quad Word	q	8
char *	Quad Word	q	8
float	Single Precision	s	4
double	Double Precision	l	8

flowplayer

CamScanner ile tarandı

Complete Memory Addressing Modes

- Most General Form

  D(Rb,Ri,S) Mem[Reg[Rb]+S*Reg[Ri]+ D]

  - D: Constant "displacement" 1, 2, or 4 bytes
  - Rb: Base register: Any of 16 integer registers
  - Ri: Index register: Any, except for `%rsp`
  - S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

- Special Cases

  (Rb,Ri) Mem[Reg[Rb]+Reg[Ri]]

  D(Rb,Ri) Mem[Reg[Rb]+Reg[Ri]+D]

  (Rb,Ri,S) Mem[Reg[Rb]+S*Reg[Ri]]

flowplayer

CamScanner ile tarandı

Address Computation Examples

%rdx	0xf000
%rcx	0x0100

D(Rb,Ri,S) **Mem[Reg[Rb]+S*Reg[Ri]+ D]**

- D: Constant "displacement" 1, 2, or 4 bytes
- Rb: Base register: Any of 16 integer registers
- Ri: Index register: Any, except for `%rsp`
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

Expression	Address Computation	Address
0x8(%rdx)	0xf000 + 0x8	0xf008
(%rdx,%rcx)	0xf000 + 0x100	0xf100
(%rdx,%rcx,4)	0xf000 + 4*0x100	0xf400
0x80(,%rdx,2)	2*0xf000 + 0x80	0x1e080

flowplayer

CamScanner ile tarandı

Understanding Arithmetic Expression Example

```
long arith
(long x, long y, long z){
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
  leaq    (%rdi,%rsi), %rax    # t1
  addq    %rdx, %rax           # t2
  leaq    (%rsi,%rsi,2), %rdx
  salq    $4, %rdx             # t4
  leaq    4(%rdi,%rdx), %rcx   # t5
  imulq   %rcx, %rax           # rval
  ret
```

Register	Use(s)
%rdi	Argument x
%rsi	Argument y
%rdx	Argument z, t4
%rax	t1, t2, rval
%rcx	t5

Compiler optimization:

- Reuse of registers
- Substitution (copy propagation)
- Strength reduction

flowplayer

CamScanner ile tarandı

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax		
0x104		
$0x108		
(%rax)		
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

©flowplayer

CamScanner ile tarandı

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)	0x11	Addr 0x10C
260(%rcx,%rdx)	0x13	Addr 0x108
0xFC(,%rcx,4)	0xFF	Addr 0x100
(%rax, %rdx,4)	0x11	Addr 0x10C

flowplayer

CamScanner ile tarandı

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov__  %eax, (%rsp)
mov__  (%rax), %dx
mov__  $0xFF, %bl
mov__  (%rsp, %rdx,4), %dl
mov__  (%rdx), %rax
mov__  %dx, (%rax)
```

flowplayer

CamScanner ile tarandı

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

mov_l_ %eax, (%rsp)
mov_w_ (%rax), %dx
mov_b_ $0xFF, %bl
mov_b_ (%rsp, %rdx,4), %dl
mov_q_ (%rdx), %rax
mov_w_ %dx, (%rax)

flowplayer

Exercises - 3

CamScanner ile tarandı

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb $0xF, (%ebx)
movl  %rax, (%rsp)
movw  (%rax), 4(%rsp)
movb %al, %sl
movq %rax, $0x123
movl  %eax, %rdx
movb %si, 8(%rbp)
```

©flowplayer

CamScanner ile tarandı

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

movb $0xF, (%ebx)	: Can not use %ebx as address register
movl %rax, (%rsp)	: Mismatch between instruction suffix and register id
movw (%rax), 4(%rsp)	: Both of the operands can not be memory
movb %al, %sl	: sl is not a register
movq %rax, $0x123	: Immediate value can not be destination operand
movl %eax, %dx	: Size of the destination operand is incorrect
movb %si, 8(%rbp)	: Mismatch between instruction suffix and register id

flowplayer

CamScanner ile tarandı

Exercises - 4

- Practice Problem 3-5
- You are given the following information. A function with prototype

*void decode1 (long *xp, long *yp, long *zp)*

is compiled into assembly code, yielding the code given or right side.

Parameters xp, yp, and zp are stored in registers %rdi, %rsi, and %rdx, respectively.

Write C code for decode1 that will have an effect equivalent to the assemby code given.

```
void decode1 (long *xp, long *yp, long *zp)
xp in %rdi, yp in %rsi, zp in %rdx
decode1:
movq (%rdi), %r8
movq (%rsi), %rcx
movq (%rdx), %rax
movq %r8, %rsi
movq %rcx, (%rdx)
movq %rax, (%rdi)
ret
```

flowplayer

CamScanner ile tarandı

Exercises - 4

void decode1 (long *xp, long *yp, long *zp)

xp in %rdi, yp in %rsi, zp in %rdx

```
decode1:
movq (%rdi), %r8        ; r8=x
movq (%rsi), %rcx       ; rcx=y
movq (%rdx), %rax       ; rax=z
movq %r8, (%rsi)        ; *yp=x
movq %rcx, (%rdx)       ; *zp=y
movq %rax, (%rdi)       ; *xp=z
ret
```

flowplayer

```
void decode1 (long *xp, long *yp, long *zp)
{
        long x = *xp;
        long y = *yp;
        long z = *zp;

        *yp = x;
        *zp = y;
        *xp = z;

}
```


CamScanner ile tarandı

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below witl formulas indicating the value that will be stored in register %rax for each of the give assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	
leaq (%rdx,%rbx), %rax	
leaq (%rdx,%rbx,3), %rax	
leaq (%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

flowplayer

CamScanner ile tarandı

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	q+3p
leaq (%rbx,%rbx,7), %rax	8p+2
leaq 0xE(,%rdx,3), %rax	3q+14
leaq 6(%rbx,%rdx,7), %rax	7q+p+6

%rbx → p
%rdx → q

wplayer

CamScanner ile tarandı

Exercises - 6

- Practice Problem 3-7
- Consider the following code, in which we have omitted the expressions being computed.

```
short scale3(short x, short y, short z) {
        short t = _____;
        return t;
}
```

Compiling the actual function with GCC yields the following assembly code:

```
short scale3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
scale3:
        leaq (%rsi, %rsi, 9), %rbx
        leaq (%rbx, %rdx), %rbx
        leaq (%rbx, %rdi, %rsi), %rbx
        ret
```

Fill in the missing expression in the C code

©flowplayer

CamScanner ile tarandı

Exercises - 6

short scale3(short x, short y, short z)

x in %rdi, y in %rsi, z in %rdx

scale3:

```
leaq (%rsi, %rsi, 9), %rbx      ; t = 9y + y =10y
leaq (%rbx, %rdx), %rbx         ; t = 10y + z
leaq (%rbx, %rdi, %rsi), %rbx   ; t = (10y + z) + y*z
ret
```

x → %rdi
y → %rsi
z → %rdx
t → %rbx

©flowplayer

CamScanner ile tarandı

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Instruction	Destination	Value
addq %rcx, (%rax)		
subq %rdx, 8(%rax)		
imul $16, (%rax, %rdx, 8)		
incq 16(%rax)		
decq %rcx		
subq %rdx,%rax		

flowplayer

CamScanner ile tarandı

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3

Destination is rax

0xFF - rdx = 0x100 - 0x3 = 0xFD

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)	0x118	0x110
incq 16(%rax)	0x110	0x14
decq %rcx	%rcx	0x0
subq %rdx,%rax	%rax	0xFD

CamScanner ile tarandı

Exercises - 8

- Practice Problem 3-10
- Consider the following code, in which we have omitted the expressions being computed.

```
short arith3(short x, short y, short z) {
        short p1 = _____;
        short p2 = _____;
        short p3 = _____;
        short p4 = _____;
        return p4;
}
```

Compiling the actual function with GCC yields the following assembly code:

```
short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:
        orq %rsi, %rdx
        sarq $9, %rdx
        notq %rdx
        movq %rdx, %rbx
        subq %rsi, %rbx
        ret
```

Fill in the missing expressions in the C code

flowplayer

CamScanner ile tarandı

Exercises - 8

```
short arith3(short x, short y, short z) {
        short p1 = __y|z__ ;
        short p2 = _p1>>9_;
        short p3 = __~p2__;
        short p4 = _p3-y__;
        return p4;
}
```

short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:

```
orq %rsi, %rdx
sarq $9, %rdx
notq %rdx
movq %rdx, %rbx
subq %rsi, %rbx
ret
```

x → %rdi
y → %rsi
z → %rdx
p4 → %rbx

p3 moves to rbx then subtruct rsi

©flowplayer

CamScanner ile tarandı

Condition Codes (Explicit Setting: Compare)

- **Explicit Setting by Compare Instruction**
  - cmpq Src2, Src1
  - Example: cmpq b,a like computing a-b without setting destination
  - CF set if carry out from most significant bit
    - (used for unsigned comparisons)
    - ZF set if a == b
    - SF set if (a-b) < 0 (as signed)
    - OF set if two's-complement (signed) overflow
    - (a>0 && b<0 && (a-b)<0) || (a<0 && b>0 && (a-b)>0)

flowplayer

CamScanner ile tarandı

Outline
Slide 1 1
Slide 2 2
Slide 3 3
Slide 4 4
Slide 5 5
Slide 6 6
Slide 7 7
Slide 8 8
Slide 9 9
Slide 10 10
Slide 11 11
Slide 12 12
Slide 13 13
Slide 14 14
Slide 15 15
Slide 16 16
Slide 17 17
Slide 18 18
Slide 19 19
Slide 20 20
Slide 21 21
Slide 22 22
Slide 23 23
Slide 24 24
Slide 25 25
Slide 26 26
Slide 27 27
Slide 28 28

Conditional Move

- **cmovX Instructions**
  - Copy the value when the move condition holds

cmovX	Condition	Description
cmove	ZF	Equal / Zero
cmovne	~ZF	Not Equal / Not Zero
cmovs	SF	Negative
cmovns	~SF	Nonnegative
cmovg	~(SF^OF)&~ZF	Greater (Signed >)
cmovge	~(SF^OF)	Greater or Equal (Signed >=)
cmovl	(SF^OF)	Less (Signed <)
cmovle	(SF^OF)|ZF	Less or Equal (Signed <=)
cmova	~CF&~ZF	Above (unsigned >)
cmovae	~CF	Above or Equal

CamScanner ile tarandı

Outline
Slide 1 1
Slide 2 2
Slide 3 3
Slide 4 4
Slide 5 5
Slide 6 6
Slide 7 7
Slide 8 8
Slide 9 9
Slide 10 10
Slide 11 11
Slide 12 12
Slide 13 13
Slide 14 14
Slide 15 15
Slide 16 16
Slide 17 17
Slide 18 18
Slide 19 19
Slide 20 20
Slide 21 21
Slide 22 22
Slide 23 23
Slide 24 24
Slide 25 25
Slide 26 26
Slide 27 27
Slide 28 28

Jumping

jX Instructions

- Jump to different part of code depending on condition codes

jX	Condition	Description
jmp	1	Unconditional
je	ZF	Equal / Zero
jne	~ZF	Not Equal / Not Zero
js	SF	Negative
jns	~SF	Nonnegative
jg	~(SF^OF)&~ZF	Greater (Signed)
jge	~(SF^OF)	Greater or Equal (Signed)
jl	(SF^OF)	Less (Signed)
jle	(SF^OF)|ZF	Less or Equal (Signed)
ja	~CF&~ZF	Above (unsigned)
jb	CF	Below (unsigned)

CamScanner ile tarandı

Reading Condition Codes

- **SetX Instructions**
  - Set low-order byte of destination to 0 or 1 based on combinations of condition codes
  - Does not alter remaining 7 bytes

SetX	Condition	Description
sete	ZF	Equal / Zero
setne	~ZF	Not Equal / Not Zero
sets	SF	Negative
setns	~SF	Nonnegative
setg	~(SF^OF)&~ZF	Greater (Signed)
setge	~(SF^OF)	Greater or Equal (Signed)
setl	(SF^OF)	Less (Signed)
setle	(SF^OF)|ZF	Less or Equal (Signed)
seta	~CF&~ZF	Above (unsigned)
setb	CF	Below (unsigned)

CamScanner ile tarandı

Exercise 1

Practise Problem 3.16

A. Write a goto version in C that performs the same computation and mimics the control flow of the assembly code, in the style shown in Figure 3.16(b). You might find it helpful to first annotate the assembly code as we have done in our examples.

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

```
  void cond(short a, short *p)
  a in %rdi, p in %rsi
cond:
  testq   %rdi, %rdi
  je      .L1
  cmpq    %rsi, (%rdi)
  jle     .L1
  movq    %rdi, (%rsi)
.L1:
  rep; ret
```

```
void cond (short a, short *p) {
    if (a == 0)
        goto done;
    if (a >= *p)
        goto done;
    *p = a;
    done:
        return;
}
```


CamScanner ile tarandı

Exercise 1
Practise Problem 3.16

B. Explain why the assembly code contains two conditional branches, even though the C code has only one `if` statement.

```
void cond(short a, short *p)
{
    if (a && *p < a)
        *p = a;
}
```

```
  void cond(short a, short *p)
  a in %rdi, p in %rsi
cond:
  testq   %rdi, %rdi
  je      .L1
  cmpq    %rsi, (%rdi)
  jle     .L1
  movq    %rdi, (%rsi)
.L1:
  rep; ret
```

If a is false (meaning, if a is zero), then the code directly jumps to the label L1 and rest of the calculations are not done!

CamScanner ile tarandı

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = _________;
    if (_________) {
        if (_________)
            val = _________;
        else
            val = _________;
    } else if (_________)
        val = _________;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

CamScanner ile tarandı

Exercise 2
Practise Problem 3.18

Starting with C code of the form

```
short test(short x, short y, short z) {
    short val = y + z - x ;
    if ( z > 5 ) {
        if ( y > 2 )
            val = x / z ;
        else
            val = x / y ;
    } else if ( z < 3 )
        val = z / y ;
    return val;
}
```

GCC generates the following assembly code:

```
  short test(short x, short y, short z)
  x in %rdi, y in %rsi, z in %rdx
test:
  leaq    (%rdx,%rsi), %rax
  subq    %rdi, %rax
  cmpq    $5, %rdx
  jle     .L2
  cmpq    $2, %rsi
  jle     .L3
  movq    %rdi, %rax
  idivq   %rdx, %rax
  ret
.L3:
  movq    %rdi, %rax
  idivq   %rsi, %rax
  ret
.L2:
  cmpq    $3, %rdx
  jge     .L4
  movq    %rdx, %rax
  idivq   %rsi, %rax
.L4:
  rep; ret
```

Fill in the missing expressions in the C code.

CamScanner ile tarandı

Exercise 3

Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = _____;
    while (_____) {
        result = _____;
        a = _____;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

CamScanner ile tarandı

Exercise 3

Practise Problem 3.24

For C code having the general form

```
short loop_while(short a, short b)
{
    short result = 0;
    while ( a > b ) {
        result = a * b + result;
        a = a - 1;
    }
    return result;
}
```

```
short loop_while(short a, short b)
a in %rdi, b in %rsi
1   loop_while:
2     movl    $0, %eax
3     jmp     .L2
4   .L3:
5     leaq    (,%rsi,%rdi), %rdx
6     addq    %rdx, %rax
7     subq    $1, %rdi
8   .L2:
9     cmpq    %rsi, %rdi
10    jg      .L3
11    rep; ret
```

We can see that the compiler used a jump-to-middle translation, using the `jmp` instruction on line 3 to jump to the test starting with label `.L2`. Fill in the missing parts of the C code.

CamScanner ile tarandı

Exercise 4

Practise Problem 3.25

For C code having the general form

```
long loop_while2(long a, long b)
{
    long result = _____;
    while (_____) {
        result = _____;
        b = _____;
    }
    return result;
}
```

```
    a in %rdi, b in %rsi
1   loop_while2:
2     testq   %rsi, %rsi
3     jle     .L8
4     movq    %rsi, %rax
5   .L7:
6     imulq   %rdi, %rax
7     subq    %rdi, %rsi
8     testq   %rsi, %rsi
9     jg      .L7
10    rep; ret
11  .L8:
12    movq    %rsi, %rax
13    ret
```

We can see that the compiler used a guarded-do translation, using the `jle` instruction on line 3 to skip over the loop code when the initial test fails. Fill in the missing parts of the C code. Note that the control structure in the assembly

CamScanner ile tarandı

Exercise 3

Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case _________:       /* Case A */
        c = _________;
        /* Fall through */
    case _________:       /* Case B */
        val = _________;
        break;
    case _________:       /* Case C */
    case _________:       /* Case D */
        val = _________;
        break;
    case _________:       /* Case E */
        val = _________;
        break;
    default:
        val = _________;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
void switcher(long a, long b, long c, long *dest)
a in %rsi, b in %rdi, c in %rdx, d in %rcx
1   switcher:
2     cmpq    $7, %rdi
3     ja      .L2
4     jmp     *.L4(,%rdi,8)
5     .section        .rodata
6   .L7:
7     xorq    $15, %rsi
8     movq    %rsi, %rdx
9   .L3:
10    leaq    112(%rdx), %rdi
11    jmp     .L6
12  .L5:
13    leaq    (%rdx,%rsi), %rdi
14    salq    $2, %rdi
15    jmp     .L6
16  .L2:
17    movq    %rsi, %rdi
18  .L6:
19    movq    %rdi, (%rcx)
20    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```

CamScanner ile tarandı

Exercise 5

Practise Problem 3.31

For a C function `switcher` with the general structure

```
void switcher(long a, long b, long c, long *dest)
{
    long val;
    switch(a) {
    case   5    :        /* Case A */
        c = a ^ 15 ;
        /* Fall through */
    case   0    :        /* Case B */
        val = c + 112 ;
        break;
    case   2    :        /* Case C */
    case   7    :        /* Case D */
        val = (c + a) <<< 2 ;
        break;
    case   4    :        /* Case E */
        val =  b    ;
        break;
    default:
        val =  a    ;
    }
    *dest = val;
}
```

GCC generates the assembly code and jump table shown in Figure 3.24.

Fill in the missing parts of the C code. Except for the ordering of case labels C and D, there is only one way to fit the different cases into the template.

```
1   void switcher(long a, long b, long c, long *dest)
2   a in %rsi, b in %rdi, c in %rdx, d in %rcx
3   switcher:
4     cmpq    $7, %rdi
5     ja      .L2
6     jmp     *.L4(,%rdi,8)
7     .section        .rodata
8   .L7:
9     xorq    $15, %rsi
10    movq    %rsi, %rdx
11  .L3:
12    leaq    112(%rdx), %rdi
13    jmp     .L6
14  .L5:
15    leaq    (%rdx,%rsi), %rdi
16    salq    $2, %rdi
17    jmp     .L6
18  .L2:
19    movq    %rsi, %rdi
20  .L6:
21    movq    %rdi, (%rcx)
22    ret
```

```
1   .L4:
2     .quad   .L3
3     .quad   .L2
4     .quad   .L5
5     .quad   .L2
6     .quad   .L6
7     .quad   .L7
8     .quad   .L2
9     .quad   .L5
```


CamScanner ile tarandı

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ________,%rdi
  4004b0: jle 4004cc
  4004b2: add ________,________
  4004b4: mov $0x0,%rdx
  4004b9: cmp ________,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc ________
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```


CamScanner ile tarandı

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc __%rdx__
  4004c1: cmp %rdi,__%rdx__
  4004c3: jl __4004bd__
  4004c5: dec __%rdi__
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

flowplayer

CamScanner ile tarandı

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case _:
      case _:
            result = ______
            break;
      case _:
            x = ______
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when y=0. Parameters x and y are passed in registers %rdi, and %rsi, respectively.

flowplayer

CamScanner ile tarandı

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case 4:
            result = x * 8;
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = x + y;
            break;
      default:
            result = x & y;
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq %rsi
     f6d:   jmp  0xf8b
     f6f:   leaq (,%rdi,8), %rsi
     f77:   subq %rdi, %rsi
     f7a:   addq %rdi, %rsi
     f7d:   jmp  0xf8b
     f7f:   addq %rdi, %rsi
     f82:   jmp  0xf8b
     f84:   shlq $3, %rdi
     f88:   movq %rdi, %rsi
     f8b:   movq %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when y=0. Parameters x and y are passed in registers %rdi, and %rsi, respectively.

flowplayer

CamScanner ile tarandı

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( _____________ )
    return ____;
  else{
    long a = _____________;
    unsigned long nx = a;
    long rv = rfun(nx);
    return ___ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

flowplayer

CamScanner ile tarandı

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( ____x==1____ )
    return _1__;
  else{
    long a = ___x>>3_____;
    unsigned long nx = a;
    long rv = rfun(nx);
    return _x_ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

flowplayer

CamScanner ile tarandı

Example 4:

The disassembled code for two functions `first` and `last` is shown below, along with the code for a call of `first` by function `main`:

```
    Disassembly of last(long u, long v)
    u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8              mov    %rdi,%rax        L1: u
3     400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4     400547:  c3                    retq                    L3: Return

    Disassembly of last(long x)
    x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7     40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8     400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9     400555:  f3 c3                 repz retq               F4: Return
          .
          .
          .
10    400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11    400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Each of these instructions is given a label, similar to those in Figure 3.27(a). Starting with the calling of `first(10)` by `main`, fill in the following table to trace instruction execution through to the point where the program returns back to `main`.

CamScanner ile tarandı

```
   Disassembly of last(long u, long v)
   u in %rdi, v in %rsi
1  0000000000400540 <last>:
2    400540:  48 89 f8              mov    %rdi,%rax        L1: u
3    400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4    400547:  c3                    retq                    L3: Return

   Disassembly of last(long x)
   x in %rdi
5  0000000000400548 <first>:
6    400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7    40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8    400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9    400555:  f3 c3                 repz retq               F4: Return
                 .
                 .
                 .
10   400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11   400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1								
F2								
F3								
L1								
L2								
L3								
F4								
M2								

CamScanner ile tarandı

```
    Disassembly of last(long u, long v)
    u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8              mov    %rdi,%rax        L1: u
3     400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4     400547:  c3                    retq                    L3: Return

    Disassembly of last(long x)
    x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7     40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8     400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9     400555:  f3 c3                 repz retq               F4: Return
            .
            .
            .
10    400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11    400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2	0X400543	imul	9	11	9	0x7fffffffe810	0X400555	Cont. last
L3	0X400547	retq	9	11	99	0x7fffffffe810	0X400555	u = u * v
F4	0X400555	retq	9	11	99	0x7fffffffe818	0X400565	Return to first
M2	0X400565	mov	9	11	99	0x7fffffffe820	--	Return to main

CamScanner ile tarandı

Example 5:

A C function `procprob` has four arguments `u`, `a`, `v`, and `b`. Each is either a signed number or a pointer to a signed number, where the numbers have different sizes. The function has the following body:

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

It compiles to the following x86-64 code:

```
1   procprob:
2     movslq  %edi, %rdi
3     addq    %rdi, (%rdx)
4     addb    %sil, (%rcx)
5     movl    $6, %eax
6     ret
```

movslq: move sign-extended long word to quad word

%rdi → 1. argument
%rsi → 2. argument
%rcx → 3. argument
%rdx → 4. argument

Determine a valid ordering and types of the four parameters. There are two correct answers.

CamScanner ile tarandı

Example 5:

A C function procprob has four arguments u, a, v, and b. Each is either a signed number or a pointer to a signed number, where the numbers have different sizes. The function has the following body:

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

It compiles to the following x86-64 code:

```
1   procprob:
2     movslq  %edi, %rdi
3     addq    %rdi, (%rdx)
4     addb    %sil, (%rcx)
5     movl    $6, %eax
6     ret
```

movslq: move sign-extended long word to quad word

%rdi → 1. argument
%rsi → 2. argument
%rdx → 3. argument
%rcx → 4. argument

a: extended from l to q (int). in %rdi (1st)
u: 8 byte value is put where it points (long*). in %rdx (3rd)
v: 1 byte value is put where it points (char*). in %rcx (4th)
b: sizeof(a) + sizeof(b) = 2 (short). in %rsi (2nd)

(int a, short b, long *u, char *v)

CamScanner ile tarandı

Example 6:

Consider a function P, which generates local values, named a0–a8. It then calls function Q using these generated values as arguments. Gcc produces the following code for the first part of P:

```
    long P(long x)
    x in %rdi
1   P:
2     pushq   %r15
3     pushq   %r14
4     pushq   %r13
5     pushq   %r12
6     pushq   %rbp
7     pushq   %rbx
8     subq    $24, %rsp
9     movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
```

A. Identify which local values get stored in callee-saved registers.

B. Identify which local values get stored on the stack.

C. Explain why the program could not store all of the local values in callee-saved registers.

CamScanner ile tarandı

Example 6:

A. Identify which local values get stored in callee-saved registers.

```
long P(long x)
x in %rdi
P:
    pushq   %r15
    pushq   %r14
    pushq   %r13
    pushq   %r12
    pushq   %rbp
    pushq   %rbx
    subq    $24, %rsp
    movq    %rdi, %rbx
    leaq    1(%rdi), %r15
    leaq    2(%rdi), %r14
    leaq    3(%rdi), %r13
    leaq    4(%rdi), %r12
    leaq    5(%rdi), %rbp
    leaq    6(%rdi), %rax
    movq    %rax, (%rsp)
    leaq    7(%rdi), %rdx
    movq    %rdx, 8(%rsp)
    movl    $0, %eax
    call    Q
```

Local values a0-a5 are saved in registers %rbx, %r15, %r14, %r13, %r12, %rbp (lines 9-14).

CamScanner ile tarandı

Example 6:

B. Identify which local values get stored on the stack.

```
long P(long x)
x in %rdi
P:
  pushq   %r15
  pushq   %r14
  pushq   %r13
  pushq   %r12
  pushq   %rbp
  pushq   %rbx
  subq    $24, %rsp
  movq    %rdi, %rbx
  leaq    1(%rdi), %r15
  leaq    2(%rdi), %r14
  leaq    3(%rdi), %r13
  leaq    4(%rdi), %r12
  leaq    5(%rdi), %rbp
  leaq    6(%rdi), %rax
  movq    %rax, (%rsp)
  leaq    7(%rdi), %rdx
  movq    %rdx, 8(%rsp)
  movl    $0, %eax
  call    Q
```

Local values a6 and a7 are stored in stack at offsets 0 and 8 relative to the stack pointer (lines 16 and 18)

player
26:49
CamScanner ile tarandı

Example 6:

C. Explain why the program could not store all of the local values in callee-saved registers.

```
long P(long x)
x in %rdi
P:
    pushq   %r15
    pushq   %r14
    pushq   %r13
    pushq   %r12
    pushq   %rbp
    pushq   %rbx
    subq    $24, %rsp
    movq    %rdi, %rbx
    leaq    1(%rdi), %r15
    leaq    2(%rdi), %r14
    leaq    3(%rdi), %r13
    leaq    4(%rdi), %r12
    leaq    5(%rdi), %rbp
    leaq    6(%rdi), %rax
    movq    %rax, (%rsp)
    leaq    7(%rdi), %rdx
    movq    %rdx, 8(%rsp)
    movl    $0, %eax
    call    Q
```

6 callee saved registers are used for a0-a5. Rest had to be stored in stack.

CamScanner ile tarandı

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	_____	_____	x_P	_____
Q	_____	_____	x_Q	_____
R	_____	_____	x_R	_____
S	_____	_____	x_S	_____
T	_____	_____	x_T	_____

©flowplayer

CamScanner ile tarandı

Array Example

Practice Problem 3.36 (solution page 377)

Consider the following declarations:

```
int    P[5];
short  Q[2];
int    **R[9];
double *S[10];
short  *T[2];
```

Fill in the following table describing the element size, the total size, and the address of element *i* for each of these arrays.

Array	Element size	Total size	Start address	Element *i*
P	4	20	x_P	$x_p + 4i$
Q	2	4	x_Q	$x_Q + 2i$
R	8	72	x_R	$x_R + 8i$
S	8	80	x_S	$x_S + 8i$
T	8	16	x_T	$x_T + 8i$

flowplayer

CamScanner ile tarandı

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_P, the address of short integer array P, and long integer index i are stored in registers `%rdx` and `%rcx`, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register `%rax` if it is a pointer and register element `%ax` if it has data type `short`.

Expression	Type	Value	Assembly code
`P[1]`			
`P + 3 + i`			
`P[i * 6 - 5]`			
`P[2]`			
`&P[i + 2]`			

flowplayer

CamScanner ile tarandı

Array Example

Practice Problem 3.37 (solution page 377)

Suppose x_p, the address of short integer array P, and long integer index i are stored in registers `%rdx` and `%rcx`, respectively. For each of the following expressions, give its type, a formula for its value, and an assembly-code implementation. The result should be stored in register `%rax` if it is a pointer and register element `%ax` if it has data type `short`.

X_p → (%rdx)
i → %rcx
Dest → %rax

Expression	Type	Value	Assembly code
`P[1]`	short	$M[x_p + 2]$	movw 2(%rdx), %ax
`P + 3 + i`	short *	$x_p + 6 + 2i$	leaq 6 (%rdx,%rcx,1), %rax
`P[i * 6 - 5]`	short	$M[x_p + 12i - 10]$	movw -10(%rdx,%rcx,12), %ax
`P[2]`	short	$M[x_p + 4]$	movw 4(%rdx), %ax
`&P[i + 2]`	short *	$x_p + 2i + 4$	leaq 4 (%rdx,%rcx,2), %rax

flowplayer

CamScanner ile tarandı

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

flowplayer

CamScanner ile tarandı

Array Example

Practice Problem 3.38 (solution page 377)

Consider the following source code, where M and N are constants declared with `#define`:

```
long P[M][N];
long Q[N][M];

long sum_element(long i, long j) {
    return P[i][j] + Q[j][i];
}
```

```
long sum_element(long i, long j)
i in %rdi, j in %rsi
1   sum_element:
2     leaq    0(,%rdi,8), %rdx
3     subq    %rdi, %rdx
4     addq    %rsi, %rdx
5     leaq    (%rsi,%rsi,4), %rax
6     addq    %rax, %rdi
7     movq    Q(,%rdi,8), %rax
8     addq    P(,%rdx,8), %rax
9     ret
```

Use your reverse engineering skills to determine the values of M and N based on this assembly code.

8 is scale value
Offset of matrix P 7i + j →
P has 7 columns

Offset of matrix Q 5j + i →
Q has 5 columns

So, M = 5 and N = 7

Compute 8i →rdx
Compute 7i →rdx
Compute 7i + j →rdx
Compute 5j →rax
Compute i + 5j →rdi
Retrieve $M[x_Q+8*(5j + i)]$ →rax
Add $M[x_p + 8*(7i+j)]$ →rax

flowplayer

CamScanner ile tarandı

Structure Example

Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = _________;
    st->p    = _________;
    st->next = _________;
}
```

A. What are the offsets (in bytes) of the following fields?

```
p:    _____
s.x:  _____
s.y:  _____
next: _____
```

B. How many total bytes does the structure require?

C. The compiler generates the following assembly code for st_init:

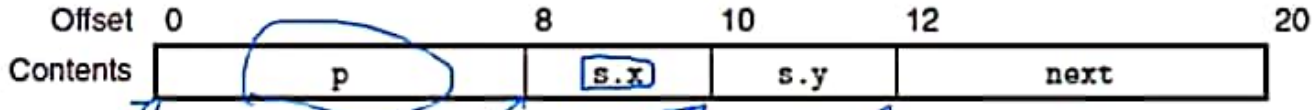
```
    void st_init(struct test *st)
    st in %rdi
1   st_init:
2     movl    8(%rdi), %eax
3     movl    %eax, 10(%rdi)
4     leaq    10(%rdi), %rax
5     movq    %rax, (%rdi)
6     movq    %rdi, 12(%rdi)
7     ret
```

On the basis of this information, fill in the missing expressions in the code for st_init.

flowplayer

CamScanner ile tarandı

Structure Example



Practice Problem 3.41 (solution page 379)

Consider the following structure declaration:

```
struct test {
    short *p;
    struct {
        short x;
        short y;
    } s;
    struct test *next;
};
```

```
void st_init(struct test *st)
st in %rdi
1   st_init:
2     movl    8(%rdi), %eax      Get st->s.x
3     movl    %eax, 10(%rdi)     Save in st->s.y
4     leaq    10(%rdi), %rax     Compute &(st->.sy)
5     movq    %rax, (%rdi)       Store in st->p
6     movq    %rdi, 12(%rdi)     Store st in st->next
7     ret
```

This declaration illustrates that one structure can be embedded within another, just as arrays can be embedded within structures and arrays can be embedded within arrays.

The following procedure (with some expressions omitted) operates on this structure:

```
void st_init(struct test *st) {
    st->s.y  = st->s.x  ;
    st->p    = &(st->s.y) ;
    st->next = st;
}
```

©flowplayer

CamScanner ile tarandı

Structure Example

Practice Problem 3.42 (solution page 379)

The following code shows the declaration of a structure of type ACE and the prototype for a function test:

```
struct ACE {
    short      v;
    struct ACE *p;
};

short test(struct ACE *ptr);
```

When the code for fun is compiled, gcc generates the following assembly code:

```
    short test(struct ACE *ptr)
    ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

flowplayer

A. Use your reverse engineering skills to write C code for test.

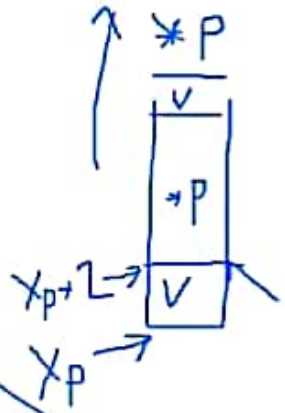
B. Describe the data structure that this structure implements and the operation performed by test

CamScanner ile tarandı

Structure Example

```
short test(struct ACE *ptr)
ptr in %rdi
1   test:
2     movl    $1, %eax
3     jmp     .L2
4   .L3:
5     imulq   (%rdi), %rax
6     movq    2(%rdi), %rdi
7   .L2:
8     testq   %rdi, %rdi
9     jne     .L3
10    rep; ret
```

result = 1
Goto middle
Loop :
  result= result * ptr->v
  ptr = ptr->p
Middle:
Test ptr
If ptr is not null, go to loop



In ACE;
v occupies 2 bytes
*p occupies 8 bytes

Ptr in %rdi
Result in %rax

```
short test(struct ACE *ptr) {
    short val = 1;
    while (ptr) {
        val *= ptr->v;
        ptr  = ptr->p;
    }
    return val;
}
```

A. Describe the data structure that this structure implements and the operation performed by test.

Each structure is an element in a singly linked list, with field v that contains the value of element and p which points to the next element.
Function fun computes the multiplication of the element values in the list.

flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

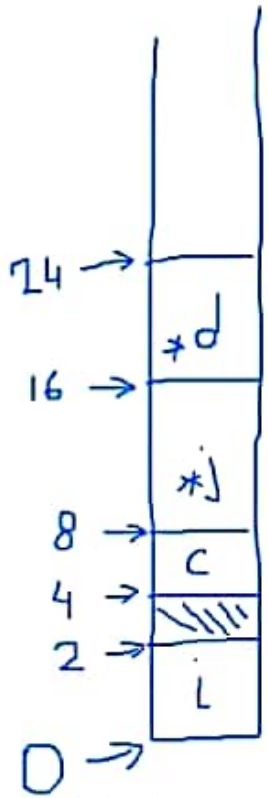
D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

A. `struct P1 { short i; int c; int *j; short *d; };`

i	c	j	d	Total	Alignment
0	2	6	14	22	8

without alignment



with alignment

flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

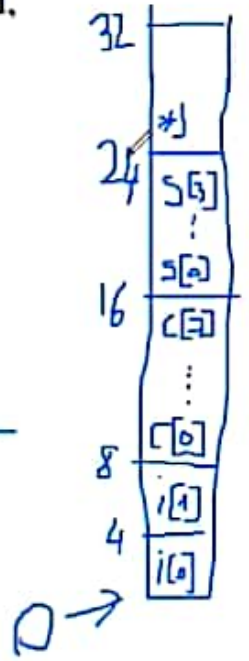
C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

i	c	s	j	Total	Alignment
0	8	16	24	32	8



flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

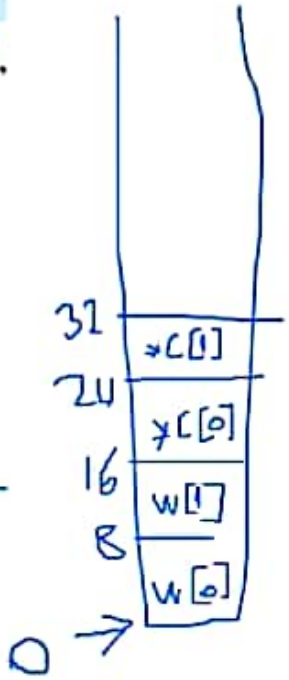
A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`



C. `struct P3 { long w[2]; int *c[2] };`

w	c	Total	Alignment
0	16	32	8

flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

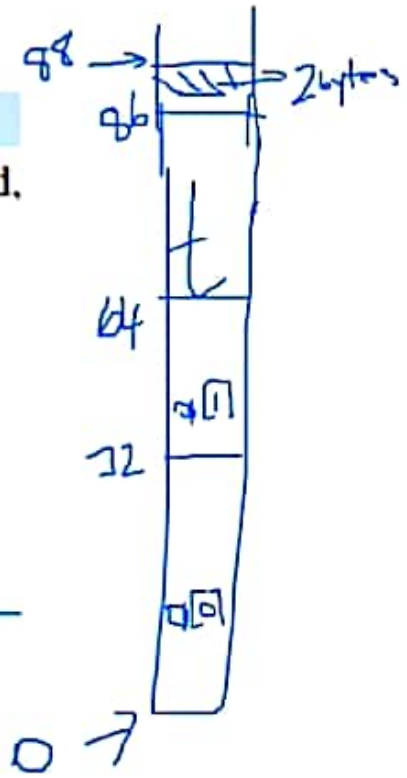
A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`



E. `struct P5 { struct P4 a[2]; struct P1 t };`

a	t	Total	Alignment
0	64	86	8

flowplayer

CamScanner ile tarandı

Structure Alignment Example

Practice Problem 3.44 (solution page 381)

For each of the following structure declarations, determine the offset of each field, the total size of the structure, and its alignment requirement for x86-64:

A. `struct P1 { short i; int c; int *j; short *d; };`

B. `struct P2 { int i[2]; char c[8]; short s[4]; long *j; };`

C. `struct P3 { long w[2]; int *c[2] };`

D. `struct P4 { char w[16]; char *c[2] };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

E. `struct P5 { struct P4 a[2]; struct P1 t };`

a	t	Total	Alignment
0	64	86	8

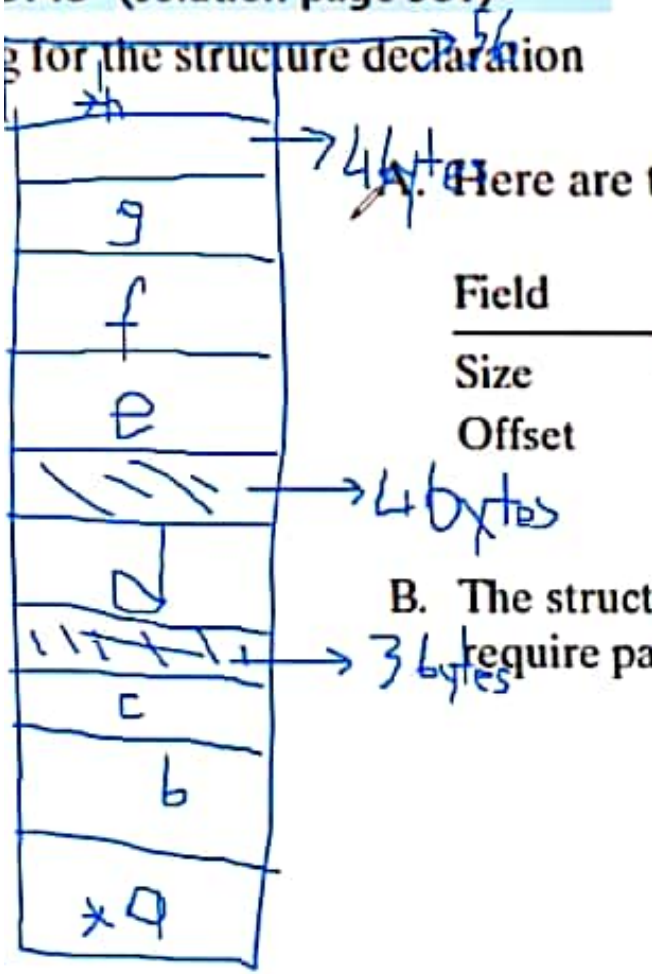
flowplayer

CamScanner ile tarandı

e Alignment Example

3.45 (solution page 381)

g for the structure declaration



A. Here are the object sizes and byte offsets:

Field	a	b	c	d	e	f	g	h
Size	8	4	1	4	8	8	4	8
Offset	0	8	12	16	24	32	40	48

B. The structure is a total of 56 bytes long. The end of the structure does not require padding to satisfy the 8-byte alignment requirement.

CamScanner ile tarandı

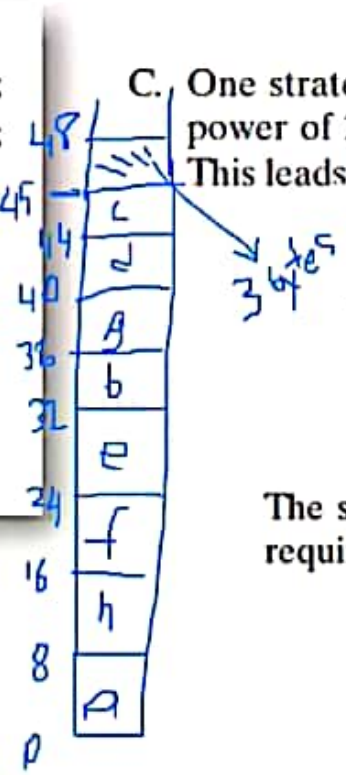
Structure Alignment Example

Practice Problem 3.45 (solution page 381)

Answer the following for the structure declaration

```
struct {
  int    *a;
  float  b;
  char   c;
  int    d;
  long   e;
  double f;
  int    g;
  char   *h;
} rec;
```

```
struct {
    int    *a;
    char   *h;
    double f;
    long   e;
    float  b;
    int    g;
    int    d;
    char   c;
} rec;
```



C. One strategy that works, when all data elements have a length equal to a power of 2, is to order the structure elements in descending order of size. This leads to a declaration:

	Field							
	a	h	f	e	b	g	d	c
Size	8	8	8	8	4	4	4	1
Offset	0	8	16	24	32	36	40	44

The structure must be padded by 3 bytes to satisfy the 8-byte alignment requirement, giving a total of 48 bytes.

flowplayer

CamScanner ile tarandı

Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```

You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that up and `dest` are loaded into registers `%rdi` and `%rsi`, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at `dest`.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

EXPR	TYPE	Code
`up->t1.u`	`long`	`movq (%rdi), %rax` `movq %rax, (%rsi)`
`up->t1.v`		
`&up->t1.w`		
`&up->t2.a`		
`up->t2.a[up->t1.u]`		
`*up->t2.p`		

CamScanner ile tarandı

Example 1

```
typedef union {
    struct {
        long   u;
        short  v;
        char   w;
    } t1;
    struct {
        int a[2];
        char  *p;
    } t2;
} u_type;
```



You write a series of functions of the form

```
void get(u_type *up, type *dest) {
    *dest =  expr;
}
```

with different access expressions *expr* and with destination data type *type* set according to type associated with *expr*. You then examine the code generated when compiling the functions to see if they match your expectations.

Suppose in these functions that up and dest are loaded into registers %rdi and %rsi, respectively. Fill in the following table with data type *type* and sequences of one to three instructions to compute the expression and store the result at dest.

Suppose you are given the job of checking that a C compiler generates the proper code for structure and union access. You write the following structure declaration:

EXPR	TYPE	Code
up->t1.u	long	movq (%rdi), %rax movq %rax, (%rsi)
up->t1.v	short	movw 8(%rdi), %ax movw %ax, (%rsi)
&up->t1.w	char *	addq $10, %rdi movq %rdi, (%rsi)
&up->t2.a	int *	movq %rdi, (%rsi)
up->t2.a[up->t1.u]	int	movq (%rdi), %rax movl (%rdi,%rax,4), %eax movl %eax, (%rsi)
*up->t2.p	char	movq 8(%rdi), %rax movb (%rax), %al movb %al, (%rsi)

CamScanner ile tarandı

Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

A. What are the offsets (in bytes) of the following fields:

e1.p _____

e1.y _____

e2.x _____

e2.next _____

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_____ = *(_____) - _____;
3   }
```

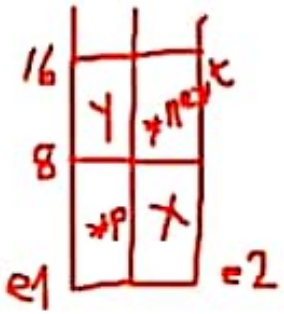
er

CamScanner ile tarandı

Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```



A. What are the offsets (in bytes) of the following fields:

e1.p	0
e1.y	8
e2.x	0
e2.next	8

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```


CamScanner ile tarandı

Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

B. How many total bytes does the structure require?

16

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->_________ = *(_________) - _________;
3   }
```

er

CamScanner ile tarandı

Example 2

Consider the following union declaration:

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next;
9       } e2;
10  };
```

C. The compiler generates the following assembly code for proc:

```
    void proc (union ele *up)
    up in %rdi
1   proc:
2     movq    8(%rdi), %rax
3     movq    (%rax), %rdx
4     movq    (%rdx), %rdx
5     subq    8(%rax), %rdx
6     movq    %rdx, (%rdi)
7     ret
```

On the basis of this information, fill in the missing expressions in the code for proc. *Hint:* Some union references can have ambiguous interpretations. These ambiguities get resolved as you see where the references lead. There is only one answer that does not perform any casting and does not violate any type constraints.

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

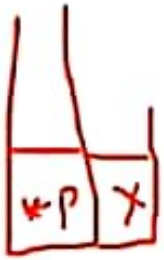
```
1   void proc (union ele *up) {
2       up->_____ = *(_____) - _____;
3   }
```

er

CamScanner ile tarandı

Example 2

```
1   union ele {
2       struct {
3           long *p;
4           long y;
5       } e1;
6       struct {
7           long x;
8           union ele *next
9       } e2;
10  };
```



C. The compiler generates the following assembly code for proc:

```
void proc (union ele *up)
up in %rdi
1   proc:
2     movq    8(%rdi), %rax  Either 1) up->e1.y or 2) up->e2.next
3     movq    (%rax), %rdx   2) Either 3) up->e2.next->e1.p or 4) up->e2.next->e2.x
4     movq    (%rdx), %rdx   3) *(up->e2.next->e1.p)
5     subq    8(%rax), %rdx  Subtract up->e2.next->e1.y
6     movq    %rdx, (%rdi)   Store in up->e2.x
7     ret
```

This declaration illustrates that structures can be embedded within unions.

The following function (with some expressions omitted) operates on a linked list having these unions as list elements:

```
1   void proc (union ele *up) {
2       up->__e2.x__ = *((__(up->e2.next)->e1.p__) - __(up->e2.next)->e1.y__);
3   }
```

er

CamScanner ile tarandı

Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

B. `double g2(int a, double *b, float *c, long d);`

C. `double g3(double *a, double b, int c, float d);`

D. `double g4(float a, int *b, float c, double d);`

er

CamScanner ile tarandı

Example 3

For each of the following function declarations, determine the register assignments for the arguments:

A. `double g1(double a, long b, float c, int d);`

Registers: a in %xmm0, b in %rdi c in %xmm1, d in %esi

B. `double g2(int a, double *b, float *c, long d);`

Registers: a in %edi, b in %rsi, c in %rdx, d in %rcx

C. `double g3(double *a, double b, int c, float d);`

Registers: a in %rdi, b in %xmm0, c in %esi, d in %xmm1

D. `double g4(float a, int *b, float c, double d);`

Registers: a in %xmm0, b in %rdi, c in %xmm1, d in %xmm2

er

CamScanner ile tarandı

Example 4

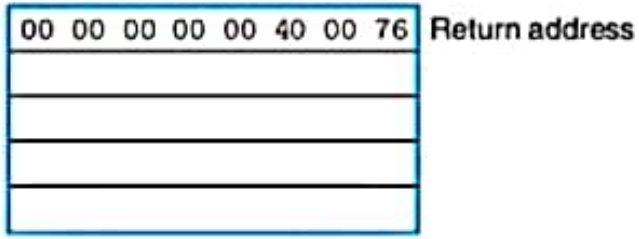
Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to 0x400776 and register %rbx equal to 0x0123456789ABCDEF. You type in the string

```
0123456789012345678901234
```

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```



(b) Disassembly up through call to gets

```
      char *get_line()
1   0000000000400720 <get_line>:
2     400720:  53                   push   %rbx
3     400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4     400725:  48 89 e7             mov    %rsp,%rdi
5     400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

CamScanner ile tarandı

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to 0x400776 and register %rbx equal to 0x0123456789ABCDEF. You type in the string

```
0123456789012345678901234
```

(a) C code

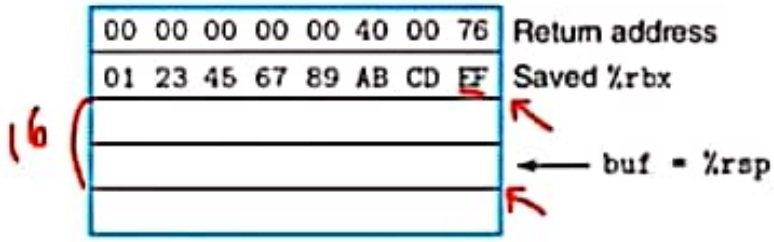
```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to gets

```
     char *get_line()
1    0000000000400720 <get_line>:
2      400720:  53                   push   %rbx
3      400721:  48 83 ec 10          sub    $0x10,%rsp
     Diagram stack at this point
4      400725:  48 89 e7             mov    %rsp,%rdi
5      400728:  e8 73 ff ff ff       callq  4006a0 <gets>
     Modify diagram to show stack contents at this point
```

A. Fill in the diagram that follows, indicating as much as you can about the stack just after executing the instruction at line 3 in the disassembly. Label the quantities stored on the stack (e.g., "Return address") on the right, and their hexadecimal values (if known) within the box. Each box represents 8 bytes. Indicate the position of %rsp. Recall that the ASCII codes for characters 0–9 are 0x30–0x39.

Stack after line 3:



The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

CamScanner ile tarandı

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to 0x400776 and register %rbx equal to 0x0123456789ABCDEF. You type in the string

0123456789012345678901234

(a) C code

```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to gets

```
      char *get_line()
1   0000000000400720 <get_line>:
2     400720:  53                   push   %rbx
3     400721:  48 83 ec 10          sub    $0x10,%rsp
      Diagram stack at this point
4     400725:  48 89 e7             mov    %rsp,%rdi
5     400728:  e8 73 ff ff ff       callq  4006a0 <gets>
      Modify diagram to show stack contents at this point
```

B. Modify your diagram to show the effect of the call to gets (line 5).

00	00	00	00	00	40	00	34	Return address
33	32	31	30	39	38	37	36	Saved %rbx
35	34	33	32	31	30	39	38	
37	36	35	34	33	32	31	30	← buf = %rsp

The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

CamScanner ile tarandı

Example 4

Figure 3.41 shows a (low-quality) implementation of a function that reads a line from standard input, copies the string to newly allocated storage, and returns a pointer to the result.

Consider the following scenario. Procedure `get_line` is called with the return address equal to `0x400776` and register `%rbx` equal to `0x0123456789ABCDEF`. You type in the string

```
0123456789012345678901234
```

(a) C code

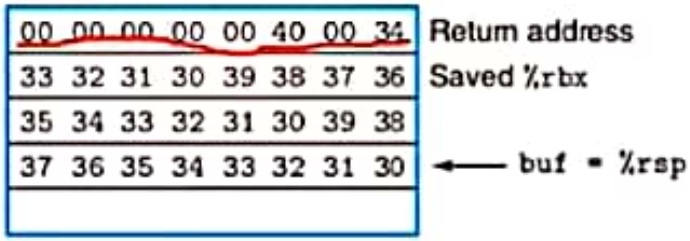
```
/* This is very low-quality code.
   It is intended to illustrate bad programming practices.
   See Practice Problem 3.46. */
char *get_line()
{
    char buf[4];
    char *result;
    gets(buf);
    result = malloc(strlen(buf));
    strcpy(result, buf);
    return result;
}
```

(b) Disassembly up through call to `gets`

```
     char *get_line()
1    0000000000400720 <get_line>:
2      400720:  53                   push   %rbx
3      400721:  48 83 ec 10          sub    $0x10,%rsp
     Diagram stack at this point
4      400725:  48 89 e7             mov    %rsp,%rdi
5      400728:  e8 73 ff ff ff       callq  4006a0 <gets>
     Modify diagram to show stack contents at this point
```

C. To what address does the program attempt to return?

The program is attempting to return to address `0x040034`. The low-order 2 bytes were overwritten by the code for character '4' and the terminating null character.



The program terminates with a segmentation fault. You run GDB and determine that the error occurs during the execution of the `ret` instruction of `get_line`.

CamScanner ile tarandı

Example 5

Stack discipline. Consider the following C code and its corresponding 32-bit x86 machine code. Please complete the stack diagram on the following page.

```
int bar (int a, int b) {
    return a + b;
}

int foo(int n, int m, int c) {
    c += bar(m, n);
    return c;
}
```

```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

er

CamScanner ile tarandı

Example 5

Draw a detailed picture of the stack, starting with the caller invoking `foo(3, 4, 5)`, and ending immediately **before** execution of the `ret` instruction in `bar`.

- The stack diagram should begin with the three arguments for `foo` that the caller has placed on the stack. To help you get started, we have given you the first one.
- Use the actual values for function arguments, rather than variable names. For example, use 3 or 4 instead of `n` or `m`.
- Always label `%ebp` and give its value when it is pushed to the stack, e.g., `%ebp: 0xffff1400`.
- You may not need to fill in all of the boxes in the diagram.

```
Value of %ebp when foo is called: 0xffffd858
Return address in function that called foo: 0x080483c9

Stack      The diagram starts with the
addresss   arguments for foo()
           +-----------------------------------+
0xffffd850 |                5                  |
           +-----------------------------------+
0xffffd84c |                                   |
           +-----------------------------------+
0xffffd848 |                                   |
           +-----------------------------------+
0xffffd844 |                                   |
           +-----------------------------------+
0xffffd840 |                                   |
           +-----------------------------------+
0xffffd83c |                                   |
           +-----------------------------------+
0xffffd838 |                                   |
           +-----------------------------------+
0xffffd834 |                                   |
           +-----------------------------------+
0xffffd830 |                                   |
           +-----------------------------------+
```

er

CamScanner ile tarandı

```
08048374 <bar>:
 8048374:       55                      push   %ebp
 8048375:       89 e5                   mov    %esp,%ebp
 8048377:       8b 45 0c                mov    0xc(%ebp),%eax
 804837a:       03 45 08                add    0x8(%ebp),%eax
 804837d:       5d                      pop    %ebp
 804837e:       c3                      ret

0804837f <foo>:
 804837f:       55                      push   %ebp
 8048380:       89 e5                   mov    %esp,%ebp
 8048382:       83 ec 08                sub    $0x8,%esp
 8048385:       8b 45 08                mov    0x8(%ebp),%eax
 8048388:       89 44 24 04             mov    %eax,0x4(%esp)
 804838c:       8b 45 0c                mov    0xc(%ebp),%eax
 804838f:       89 04 24                mov    %eax,(%esp)
 8048392:       e8 dd ff ff ff          call   8048374 <bar>
 8048397:       03 45 10                add    0x10(%ebp),%eax
 804839a:       c9                      leave
 804839b:       c3                      ret
```

foo(3,4,5) -> bar(4,3)

ebp = 0xffffd840
Return address: 0x080483c9
esp = 0xffffd834
eax = 4 + 3 = 7
Return address: 0x8048397

Stack address	The diagram starts with the arguments for foo()
0xffffd850	5
0xffffd84c	4
0xffffd848	3
0xffffd844	caller ra: 0x080483c9
0xffffd840	old ebp: ffffd858
0xffffd83c	3
0xffffd838	4
0xffffd834	foo ra: 0x08048397
0xffffd830	ebp ...-8 4 0
0xffffd82c	
0xffffd824	

ebp
esp

CamScanner ile tarandı